

Pricing Deception: Deciding When a Web Attack Detector Should Probe

Parth Kansal, Amrit Singh Nijjar, Siddhant Mehta, and Niyati

Chandigarh University, Punjab, India

parth.kansal823@gmail.com, amrit.nijj814@gmail.com,
siddmehta21@gmail.com, kamranियति@gmail.com

Abstract. A web application firewall judges each request based on its content. Deception is usually done after a session has already been judged hostile. Our approach uses it while the decision is uncertain. When system is not sure, it adds invisible probes to response. These are not shown in browser but the client that acts on it reveals itself. The decision to probe is based on the value of information it provides rather than manual thresholds. Under the usual cost model, there is no middle option between passing and diverting. The probing band appears only once the information is priced. In randomised comparison over 99 runs, probing increases the share of attacks caught by 7.0 percentage points and no harmless sessions was diverted.

Keywords: Cyber deception · Intrusion detection · Web application security · Value of information · Cost-sensitive learning · Honeytokens

1 Introduction

Accepting or rejecting a request is in the hands of the firewall. It makes decision on the information the request carries. Act early on weak evidence can block real user. If wait for strong evidence, the attacker has already sent several requests. It makes an trade off. Learned detectors [16, 24] solve the problem to a degree, but they still watch, wait, and then commit.

Some attacks make it worse. A request for `/records/5` looks similar for `/records/6`. If that record belongs to someone else, the request is an attack. There is no pattern to match for this, so a signature firewall is blind. A learned detector also struggles because an reporting tool walks the same ids in the same order, and nothing in the traffic separates the two.

Deception is the route around that blind spot [17, 18], but most systems use it after the decision is taken: a session is deemed hostile, and the response to it is the activation of a honeypot. We put the deception before that decision is taken. While a session is still in doubt, its replies would carry something extra. A browser would never show it, but anyone reading the raw response will see it. It can be a fake database error naming a table that does not exist, or an unused field in a JSON reply. We call these additions *probes*. An honest user will not notices, while a attacker that acts on that probe reveals itself.

A probe can be engineered. Our objective is to find out *when* to use that probe. A probe buys no protection, because the request still reaches the application. Its only value is the information it might reveal, and that value can be priced [10].

Contributions. This paper makes two contributions.

1. **A priced third action.** Probing costs more than passing at every belief, so under ordinary cost accounting there is no band for a threshold to create. The band appears only once the information a probe buys is priced, which makes its edges follow from the cost table rather than from a choice of ours.
2. **A measured causal effect.** Inside the probing system, we withhold the probe from one session in ten, chosen at random, so its effect is a randomised estimate rather than a comparison of two different designs.

2 Literature Review

We group previous work into five areas and find the gaps in them.

Detecting web attacks. Signature firewalls matches known bad patterns. Thus attacks written in valid syntax can bypass it. Also, known payloads can be changed to avoid a match. Learned detector replaces fixed patterns with a model. Kruegel and Vigna [16] built early anomaly detector for web request. Robertson et al. [19] apply this to multiple sites. Later, adaptive [2] and deep [24] method was applied. All these systems observe session, collect evidence and make decision. Our system follow this and is not our contribution.

Detect automated clients. Next, is to detect client is automated rather than finding if it is harmful. Iliou et al. [11] tells about advance bots. These can behave same as real browsing which make difficult to distinguish them from automated bot. Thus, we measure automation and hostility separately. An automated scanner is both automated and hostile, price comparison bot is only automated. A human attacker is hostile but not automated. Thus, a single score can't distinguish between these cases.

Deception and honeypots. Deception is widely studied as defense. Provos [18] built an early virtual honeypot framework. Almeshekeh and Spafford [1] suggests that it should be planned around target belief. Pawlick et al. [17] classified different deception approaches. Beltrán-López et al. [5] reviewed recent deception research and Ferguson-Walter et al. [9] studied how deception affects attackers and found that this can disrupt their actions. In all these works deception is a place where caught attacker is moved. Tripwires are been used for long time, including honeyfiles [27], decoy documents [6], fake passwords [12]. These are usually kept active always. Kahlhofer and Rass [15] reviewed different ways to deploy inside application without access of source code. Kahlhofer et al. [14] automate the deploy and rotating trap, but didn't tell when to place. These traps can be detected. Honeypots [26] and honeytokens [23] can be detected using fingerprinting techniques.

Generated fake environments. Language models can be used to fabricate a system that can be presented as a real one to the user. Sladić et al. [21] dynamically create shell output. Bridges et al. [7] summarise the area and conclude that the progress made so far is limited. Such work measures how enticing a planted item is [13], and Vero et al. [25] add stealth and fidelity. None of it asks if a fake system contradicts itself during a long visit. This is what we test. Our fake site is also constructed such that its content generator can be swapped, which lets us check that the result is not dependent on the production of the content.

Decision theory. The mathematics that underlies our approach is standard. The value of sample information was introduced by Howard [10]. Elkan [8] described cost-sensitive decisions using a loss matrix, and Axelsson [4] showed how the rarity of attacks limits the performance of intrusion detectors. There is also work that looks at transforming the output of the classifiers to provide probabilities [28]. There is no action in any of them leading to the purchase of evidence. This is the gap. There is no new mathematics here. We can only add to it by recognising a probe as a purchase of information, and by pricing it to turn the hand-chosen option into a calculated choice.

3 Methodology

3.1 System Overview

Our system is reverse proxy in front of real web application. Client only talk to proxy. Each request goes to application through proxy only. This let application never knows that deception exists.

Each request follows the five steps.

1. Identify the session using a cookie.
2. Turn the request into 18 features.
3. Update two scores of the session.
4. Price the three possible actions and take the cheapest.
5. Write the decision to a untampered log

The three actions are *pass*, *probe* and *divert*. Pass send the request to real application, Divert moves session to the fake copy and probe adds probe to response and passes the request on unchanged.

3.2 Two Scores

The 18 features are split into two groups because threat has two parts. There are ten *automation features* which tells how the client behaves. This include time between request, how regular timing is, whether static assets were fetched, how complete headers are and whether cookie was sent. The eight *hostility features* describes what client is trying to do. It measure unusual character in user input, failed logins, database keyword in suspicious position, ratio of errors to

Table 1. The fixed cost table.

	pass probe divert		
client is harmless	0	1	200
client is hostile	25	25	-20

replies and how many different account the session tried to reach. We used two logistic models which turn these features into two scores. Both scores are build up as session continues. We kept them separate because automated client can be completely harmless. So, hostility tells whether we should take action and automation score helps decide which probe to use, since human and automated client respond different to different bait.

The belief is not a calibrated probability, its a score. On held out data calibration error is 0.157. This is been reduced to 0.018 by using isotonic map [28]. We haven't apply this map because under fixed cost table it performs worse. The gain in recall comes from diverting harmless clients. The bands derived below treat the score as probability. Section 5 discusses the drawback of this assumption.

3.3 Choosing Action by Price

The system continuously maintain a belief p whether client is hostile. Each action has a cost, defined by loss matrix [8]. We hashed and fixed these cost before collecting data so no threshold derived from them could be adjusted afterwards to improve results. The cost used in our system are shown in Table 1.

Three numbers in Table 1 tells main trade offs. Diverting harmless client is worst decision, so it costs 200. Probing an hostile client and passing it through are both set to 25 as the request still go to real application in either case. Probing harmless client adds small extra cost of 1.

Using these, the expected cost of each action is straight line in p :

$$\mathbb{E}[C_{\text{pass}}] = 25p, \quad \mathbb{E}[C_{\text{probe}}] = 24p + 1, \quad \mathbb{E}[C_{\text{divert}}] = -220p + 200. \quad (1)$$

Compare first two terms of Equations (1):

$$\mathbb{E}[C_{\text{probe}}] - \mathbb{E}[C_{\text{pass}}] = 1 - p \geq 0. \quad (2)$$

The gap in Equation (2) is $1 - p$. This can never be negative. This means probing is never cheaper than passing. It would only choose between passing and diverting. The switch between two occurs when $25p = 200 - 220p$, at $p \approx 0.816$. This may seems probing band was chosen manually but this is not the case. At this stage, there is no band to choose.

3.4 Pricing the Information

Probe is useful because of the information provided. It has two outcomes: client takes bait or it does not. This outcome gives extra information about the client. We measure this with expected value of sample information [10]:

$$V(p) = \min_a \mathbb{E}[C(a) | p] - \mathbb{E}_Z \left[\min_a \mathbb{E}[C(a) | p'(Z)] \right]. \quad (3)$$

In Equation (3), Z is what we observe and $p'(Z)$ is updated belief after seeing it. Let β_a be probability a hostile client takes the bait, and β_b be probability a harmless client does. Thus, the probability of bite is $p\beta_a + (1-p)\beta_b$ and we update our belief using Bayes' rule. Every term in Equation (3) is straight line, so $V(p)$ can be calculated directly. No need of simulation. $V(p)$ prices one probe at a time. We do not plan a sequence of them.

Earlier probe was never worth taking but now with information priced, its worth taking. So probe real value is cost minus value of the information. The system now picks cheapest of all three actions:

$$\text{price}(\text{probe}) = 1 + 24p - V(p). \quad (4)$$

Two things fix the band at two end. First, $V(p) \geq 0$ always. The smallest of several straight lines is concave. Jensen's inequality tells us that learning from the probe cannot increase the expected cost. Thus, information can only help or leave the decision unchanged. Second, $V(0) = V(1) = 0$: when we already know that client is hostile or harmless, probe can't change the decision. So, probing band is away from both end. The width is determined by cost and measured bite rate. It is not set by manual chosen threshold.

Equation (4) gives:

$$\text{pass if } p < 0.065, \quad \text{probe if } 0.065 \leq p < 0.879, \quad \text{divert if } p \geq 0.879. \quad (5)$$

Equation (5) changes the divert point. It pushes the point up from 0.816 to 0.879. System that can probe waits longer than one that cannot. We did not aim for this. This value is from the calculation.

3.5 Measuring the Probe and Tools Used

The calculation above need one measured input i.e. how often each type client takes bait. We measured this separately after probe is been built. We locked the values before evaluation began. For main probe, 138 out of 183 hostile session took bait while none of 74 harmless sessions did. We converted these counts to rates using posterior mean with Jeffreys Beta($\frac{1}{2}, \frac{1}{2}$) prior. It gives $\beta_a = 0.753$ and $\beta_b = 0.0067$. The smoothing is important for β_b . Without this, zero bites observation would produce infinite likelihood ratio and would suggest that harmless client can never take bait, which is strong conclusion to draw from 74 sessions.

All detection code, features, decision rule and experiments are our own written in Python. No language model is used in the request path or for data processing, filtering, visualisation or proving any result. Language models are used in three places.

- **Simulated attackers:** We use three local models – Llama 3.2 1B, Llama 3.2 3B, and Qwen 2.5 7B. This help to simulate the attack. Each model read the reply and chose next request without being told that probes exist. This test our assumption how curious attacker might interact with system.
- **Content generation:** It help to generate content of fake site, allow us to test if the consistency layer depend on how content is created. When consistency layer was enabled, the fake site shows no contradiction across all 15 entities. Without the layer, all 15 entities contains contradictions.
- **Adversarial testing:** We attacked our own system using language model and look for weaknesses. This help to uncover several flaws in fake site, which we fixed and reported in Section 5.

3.6 Experimental Setup

We compare three systems:

- **B1 (b1_rules):** a signature firewall using fixed patterns.
- **B2 (b2_passive):** our detector without probes. It can only pass or divert.
- **B4 (b4_full):** the full system, with probes and the fake site.

B2 and B4 are same codes and same locked model. Configuration flag selects which of the two runs. B1 is independent signature baseline. We also evaluated two more systems which are not been reported as separate arms. Those are undefended arms and B3. B3 is based on approach from prior work. It cannot produce different values from B2 for both reported metrics because decoy only changes the response seen by diverted session and does not change the diversion decision.

We ran each system on 99 independent rounds of generated traffic. Each contains 120 attack sessions and 80 harmless sessions. This gives total of 11,880 attack sessions and 7,920 harmless sessions per system. In one round, all three system gets the same traffic so that attack session can be compared. This is 60% attack rate, far more than any real deployment. The band edges do not depend on it. They are determined by cost table and bite rates. How often a session reaches the band does, so our result is measured under a prevalence no production site would see.

Throughout, *diverted* means system had decided that client is hostile. We applied this across all system, so that numbers are comparable. Our evaluation follows the evaluation guidance for machine learning in security [3, 22]: comparison are paired, costs are fixed and attack rounds used for training and testing the detector are kept separate.

4 Results

4.1 What the Probe Itself Causes

B2 and B4 are two systems which differ in the probe and in the divert point that offering it moves. To isolate effect of probe, we ran comparison inside B4.

Table 2. The randomised comparison inside the full system.

group	sessions caught	rate
probe sent	10,643	10,112 0.950
probe withheld	1,237	1,089 0.880

Table 3. Attack recall and harmless sessions diverted, over 99 rounds.

system	attack recall (95% CI)	harmless diverted
B1 signature firewall	0.366 [0.358, 0.375]	0 / 7,920
B2 detector only	0.889 [0.883, 0.894]	4 / 7,920
B4 full system	0.943 [0.939, 0.947]	0 / 7,920

We randomly select one in ten sessions that reaches the probing band and did not show them a probe. Both had same belief state and follows same rule. Only difference was if a probe was sent or not. Result are shown in Table 2.

Table 2 shows a difference of **7.0 percentage points**. The 95% confidence interval is [5.2, 8.8], and Fisher’s exact test gives $p = 3.4 \times 10^{-19}$. Since two group were randomly separated at the same belief state, the probe caused the difference.

We also check for the random split balance. The largest difference in composition of two group across the five attack types is 1.2 percentage points. A chi-square test gives 2.58 with four degrees of freedom, which is below 9.49 threshold. This tells that split was well balanced. An uneven split may have introduced differences due to attack type mix rather than effect of probe.

4.2 Comparison With Baselines

Table 3 shows that B4 performs better than B2 on both recall and harmless diversions. The recall ranges for both systems are not overlapping. When we compare the same session in both systems, B4 catches 842 attacks that B2 misses, while B2 catches 197 that B4 misses. A paired test gives $p = 1.9 \times 10^{-95}$. B4 perform better in all 99 rounds.

Those 197 that B4 misses are delays, not noise. Their belief score was between 0.816 and 0.879, where B2 diverts immediately while B4 first asks a question. Some of the session ended before question is answered. That is the real cost of having a third action.

Both zero values should be explained. B1 rely on suspicious syntax that are not seen in harmless traffic. B4 result is interesting. All four harmless sessions that B2 divert wrongly were probed by B4. None of these took the bait. Their lack of response kept their belief below 0.879, while B2 had diverted them at 0.816. Four session are too few to conclude that B4 is safer but result tells how the mechanism can reduce unnecessary diversions.

Table 4. OWASP ModSecurity Core Rule Set on the same traffic. The replay uses a smaller sample, so these intervals are wider.

setting	attack recall (95% CI)	harmless sessions blocked
1 (default)	0.353 [0.305, 0.404]	0 / 240
2 and 3	0.544 [0.493, 0.595]	0 / 240
4 (maximum)	1.000 [0.989, 1.000]	72 / 240 (30%)

Table 5. Recall by attack type, over the same 99 rounds.

attack type	sessions	B2	B4	gain
Obfuscated SQL injection	3,960	0.898	0.921	+0.022
Slow SQL injection	1,980	1.000	1.000	0.000
Record walking via API	1,980	0.994	0.993	-0.001
Password spraying	1,980	1.000	1.000	0.000
Record walking via web pages	1,980	0.540	0.823	+0.282

4.3 Comparison With a Real Firewall

B1 is our own firewall. So we wanted to check its performance independently. We replayed same traffic using OWASP ModSecurity Core Rule Set. It is widely used rule set for web applications. Table 4 shows the result for all four settings, two of which produce the same result.

At its default setting the rule set scores 0.353, which is just below B1’s score of 0.366, so B1 is not a weak baseline. But the limit is not a tuning limit. The settings that leave harmless clients alone top out near 0.54, while the rule set reaches perfect recall only by blocking 30% of harmless clients, which no real site could accept. Our system diverts nobody and achieves 0.943. One thing in favour of the rule set is that it is assessed on blocking any one request, whereas our systems are assessed on identifying the entire session, which is a harder task.

4.4 Where the Probe Helps

Table 5 displays the whole gain in one row. Walking through records using the normal web pages improves from 0.540 to 0.823. Everywhere else the probe changes almost nothing. This is exactly the situation a signature cannot see and the detector cannot settle, and where the pricing says information is worth buying.

One attack type is only marginally improved, and it accounts for about half of what the complete system is unable to capture. Obfuscated SQL injection sessions do act on the bait: 18% of the sessions that were shown a probe took it. The detector manages to settle most of these without assistance, so recall rises only from 0.898 to 0.921. An obfuscated payload leaves a trace in the request itself, so the belief is seldom in doubt, and a probe that confirms what the meter already knows has no effect on the decision. The gain is small due to the lack of uncertainty remaining.

5 Discussion

Our results are consistent with the mechanism and not just a number. If the probe had raised recall throughout, it would indicate a poor detector, not a proper calculation. But rather, the gain is sitting where the decision is in doubt, where the pricing indicates. The safety result looks like the same thing. Its divert point is not something we introduced, and its job was to fix all the mistakes made by the simpler detector.

Where our approach does not win. If expected cost is the only measure, it is better to set the band edges by hand than to compute them. More than half of decisions can be made using two belief values, and the additional precision of a calculated edge does not really help. All settings that perform better than it do so by diverting harmless clients. Our calculated edges are the best that can be obtained in settings that do not divert anyone.

Adversarial testing of the fake site. A language model agent attacking our own system (Section 3.5) discovered four ways to tell the fake site from the real one, all of which were found by exploiting the fix for the previous method. A page returned other values following a diversion. A list page named a person differently from their own profile page. After a diversion there was a record which had not existed before. The bogus dashboard showed another user’s records as the client’s own. Then a human subject found a fifth, two profiles with the same name but different information. All five are fixed. The lesson generalises. A fake site agreeing with itself is not the same as agreeing with what the attacker saw. That is a different failure from the fingerprinting of deployed decoys [23, 26], which reads the artefact instead of its history.

Do people notice the fake site? In a small-scale blind study, participants were randomly divided into two arms, the real and the fake application. The assignments were done in a block-randomised fashion with a fixed seed. One of the two called the fake site a mock-up, and one of the two called the real application a mock-up. The difference is zero and Fisher’s exact test gives $p = 1.0$, but no p below 0.333 was reachable at this size, so that number means nothing by itself. Showing the two arms are alike is an equivalence question [20] and requires the eight participants our protocol asks for. The study does provide a baseline. One of the participants who studied the *genuine* application called it a “mock-up” because of the way the staff directory was organised, putting together three departments and leaving a fourth one out. Real systems are called fake by people because of an ordinary layout quirk, and the correct baseline is not zero. This is reported as a pilot and not a result.

Who actually takes the bait? This is not a steady increase of bite rate as the attacker’s ability grows, and that is what bounds our central claim. When it gets stronger in the three local models it rises from 0.077 to 0.556. At both extremes outside, it is equal to zero. Replies were never read by the off-the-shelf tools.

Rather, our human tester and a frontier-model agent have never taken a single bait: there were two interactive sessions and eighteen probes on display, none of which were touched. Both were caught by the injection path rather than by a probe. The middle of the range is the only range in which we have corroborating evidence for our gain of 7.0 points. The attractiveness of a planted item can be measured [13], and it is a natural next step to calibrate our probes to a human panel. In the end, this narrows the mechanism without removing it.

What if the attacker is aware of the probe? What happens once the adversary learns to ignore probes is an inescapable question. If you give an additional action it will raise the divert point, and an attacker who maintains the belief in the band and never bites will be probed for as long as possible instead of being diverted. This is answered cohesively by the pricing without adding another term. If a session is asked the question n times, it has already answered it. Its expected information is therefore discounted by $(1 - \beta_a)^n$, that is the probability that a hostile client refuses that many times consecutively. As this discount falls, probing ceases to be the most economical move and the rule approaches the two-action comparison it would have with no probing. It is an asymptotic guarantee and not a per-session guarantee. At $n = 0$ the discount is exactly 1, so all the bands above are unchanged.

Limitations. Four, in order of importance. First, all our traffic is generated, so all our rates refer to our traffic and would need to be measured again for real traffic; the calculation and the randomised design are independent of that. Second, we tune and test on one application, and the locked detector is still working in front of another, totally different application, but the probes themselves would have to be rewritten. Third, the number of people who have tried the fake site is only four. Fourth, the bite rate is based on a notion of curiosity that no human we have observed has yet borne out.

6 Conclusion

Passive defence requires the defender to wait for evidence, and that leaves a dilemma. We have demonstrated that, in fact, the defender can generate the evidence by sending a small, invisible probe in a reply. The value of the information provided by the probe determines when to insert the probe, not a threshold set by the user. Our measurements confirm this: The gain is seen only when the detector is really uncertain, and a randomised holdout within the probing system shows that the probe caused the improvement. There was no loss of harmless session in 99 runs. What is left is to test the idea with real traffic and real attackers.

References

1. Almeshekah, M.H., Spafford, E.H.: Planning and integrating deception into computer security defenses. In: Proceedings of the 2014 New Security Paradigms Work-

- shop. p. 127–138. NSPW '14, Association for Computing Machinery, New York, NY, USA (2014). <https://doi.org/10.1145/2683467.2683482>
2. Amouei, M., Rezvani, M., Fateh, M.: Rat: Reinforcement-learning-driven and adaptive testing for vulnerability discovery in web application firewalls. *IEEE Transactions on Dependable and Secure Computing* **19**(5), 3371–3386 (Sep 2022). <https://doi.org/10.1109/TDSC.2021.3095417>
3. Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L., Rieck, K.: Dos and don'ts of machine learning in computer security. In: *Proceedings of the 31st USENIX Security Symposium*. pp. 3971–3988. Boston, MA, USA (2022)
4. Axelsson, S.: The base-rate fallacy and the difficulty of intrusion detection. *ACM Trans. Inf. Syst. Secur.* **3**(3), 186–205 (Aug 2000). <https://doi.org/10.1145/357830.357849>
5. Beltrán-López, P., Gil Pérez, M., Nespoli, P.: Cyber deception: Taxonomy, state of the art, frameworks, trends, and open challenges. *IEEE Communications Surveys & Tutorials* **28**, 1520–1556 (2026). <https://doi.org/10.1109/COMST.2025.3594788>
6. Bowen, B.M., Hershkop, S., Keromytis, A.D., Stolfo, S.J.: Baiting inside attackers using decoy documents. In: Chen, Y., Dimitriou, T.D., Zhou, J. (eds.) *Security and Privacy in Communication Networks*. pp. 51–70. Springer Berlin Heidelberg, Berlin, Heidelberg (2009)
7. Bridges, R.A., Mitchell, T.R., Muñoz, M., Henriksson, T.: Sok: Honeypots & llms, more than the sum of their parts? In: *2026 IEEE 11th European Symposium on Security and Privacy (EuroS&P)*. pp. 910–928 (July 2026). <https://doi.org/10.1109/EuroSP68448.2026.00063>
8. Elkan, C.: The foundations of cost-sensitive learning. In: *Proceedings of the 17th International Joint Conference on Artificial Intelligence - Volume 2*. p. 973–978. IJCAI'01, Morgan Kaufmann Publishers Inc., San Francisco, CA, USA (2001)
9. Ferguson-Walter, K.J., Major, M.M., Johnson, C.K., Muhleman, D.H.: Examining the efficacy of decoy-based and psychological cyber deception. In: *Proceedings of the 30th USENIX Security Symposium* (2021)
10. Howard, R.A.: Information value theory. *IEEE Transactions on Systems Science and Cybernetics* **2**(1), 22–26 (Aug 1966). <https://doi.org/10.1109/TSSC.1966.300074>
11. Iliou, C., Kostoulas, T., Tsikrika, T., Katos, V., Vrochidis, S., Kompatsiaris, I.: Detection of advanced web bots by combining web logs with mouse behavioural biometrics. *Digital Threats* **2**(3) (Jun 2021). <https://doi.org/10.1145/3447815>
12. Juels, A., Rivest, R.L.: Honeywords: making password-cracking detectable. In: *Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security*. p. 145–160. CCS '13, Association for Computing Machinery, New York, NY, USA (2013). <https://doi.org/10.1145/2508859.2516671>
13. Kahlhofer, M., Achleitner, S., Rass, S., Mayrhofer, R.: Honeyquest: Rapidly measuring the enticingness of cyber deception techniques with code-based questionnaires. In: *Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses*. p. 317–336. RAID '24, Association for Computing Machinery, New York, NY, USA (2024). <https://doi.org/10.1145/3678890.3678897>
14. Kahlhofer, M., Golinelli, M., Rass, S.: Koney: A cyber deception orchestration framework for kubernetes. In: *2025 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. pp. 690–702 (June 2025). <https://doi.org/10.1109/EuroSPW67616.2025.00084>

15. Kahlhofer, M., Rass, S.: Application layer cyber deception without developer interaction. In: 2024 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW). pp. 416–429 (July 2024). <https://doi.org/10.1109/EuroSPW61312.2024.00053>
16. Kruegel, C., Vigna, G.: Anomaly detection of web-based attacks. In: Proceedings of the 10th ACM Conference on Computer and Communications Security (CCS '03). pp. 251–261 (2003). <https://doi.org/10.1145/948109.948144>
17. Pawlick, J., Colbert, E., Zhu, Q.: A game-theoretic taxonomy and survey of defensive deception for cybersecurity and privacy. *ACM Comput. Surv.* **52**(4) (Aug 2019). <https://doi.org/10.1145/3337772>
18. Provos, N.: A virtual honeypot framework. In: Proceedings of the 13th USENIX Security Symposium. pp. 1–14. San Diego, CA, USA (2004)
19. Robertson, W., Vigna, G., Kruegel, C., Kemmerer, R.A., et al.: Using generalization and characterization techniques in the anomaly-based detection of web attacks. In: NDSS (2006)
20. Schuirmann, D.J.: A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. *Journal of pharmacokinetics and biopharmaceutics* **15**(6), 657–680 (1987)
21. Sladić, M., Valeros, V., Catania, C., Garcia, S.: Llm in the shell: Generative honeypots. In: 2024 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW). pp. 430–435 (July 2024). <https://doi.org/10.1109/EuroSPW61312.2024.00054>
22. Sommer, R., Paxson, V.: Outside the closed world: On using machine learning for network intrusion detection. In: 2010 IEEE Symposium on Security and Privacy. pp. 305–316 (May 2010). <https://doi.org/10.1109/SP.2010.25>
23. Srinivasa, S., Pedersen, J.M., Vasilomanolakis, E.: Towards systematic honeypot fingerprinting. In: 13th International Conference on Security of Information and Networks. SIN 2020, Association for Computing Machinery, New York, NY, USA (2021). <https://doi.org/10.1145/3433174.3433599>
24. Tekerek, A.: A novel architecture for web-based attack detection using convolutional neural network. *Computers & Security* **100**, 102096 (2021). <https://doi.org/10.1016/j.cose.2020.102096>
25. Vero, M., Kaczmarczyk, F., Petrov, I., Shumailov, I., Hayes, J., Heinen, N., Fan, T., Invernizzi, L., Vechev, M.: Honeyval: A comprehensive evaluation framework for llm-powered http honeypots (2026), <https://arxiv.org/abs/2605.29963>
26. Vetterl, A., Clayton, R.: Bitter harvest: systematically fingerprinting low- and medium-interaction honeypots at internet scale. In: Proceedings of the 12th USENIX Conference on Offensive Technologies. p. 9. WOOT'18, USENIX Association, USA (2018)
27. Yuill, J., Zappe, M., Denning, D., Feer, F.: Honeyfiles: deceptive files for intrusion detection. In: Proceedings from the Fifth Annual IEEE SMC Information Assurance Workshop, 2004. pp. 116–122 (June 2004). <https://doi.org/10.1109/IAW.2004.1437806>
28. Zadrozny, B., Elkan, C.: Transforming classifier scores into accurate multi-class probability estimates. In: Proceedings of the Eighth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. p. 694–699. KDD '02, Association for Computing Machinery, New York, NY, USA (2002). <https://doi.org/10.1145/775047.775151>